Informe final — TP8 Food Store: cierre de la capa de datos

Materia: Base de Datos II — Tecnicatura Universitaria en Programación
Proyecto: Food Store — PostgreSQL (servidor, fuente de verdad) + MongoDB (proyección de lectura) + SQLite (tablet móvil)
Fecha: 2026-11-01

1. Objetivo y proyecto

Este informe cierra la capa de datos del sistema Food Store. El proyecto opera en tres escenarios de base de datos sobre un mismo negocio: **PostgreSQL** como sistema transaccional del servidor y única fuente de verdad (200.000 pedidos, 700.410 detalles, 50.000 productos, 20.000 clientes); **MongoDB** como proyección de lectura del subconjunto pedidos + detalle (5 pedidos de prueba, líneas embebidas `lineas[]`); y **SQLite** en la tablet de venta en feria, que trabaja sin señal y guarda una copia mínima del catálogo más borradores de venta.

> *"El móvil propone, el servidor dispone. La tablet solo consulta y arma borradores. Precio, stock y ventas los decide PostgreSQL. `SIMULADO_APROBADO` no es una venta."*

Qué hace cada motor

Motor	Rol	Qué guarda
PostgreSQL	**Fuente de verdad** (ACID)	Clientes, categorías, productos, pedidos, ventas confirmadas
MongoDB	Proyección de lectura	Colección `pedidos` con líneas embebidas; consultas sin `JOIN`
SQLite (tablet)	Copia + borradores	`producto_cache`, `pedido_borrador`, `item_borrador`, `cola_sync` (4 tablas STRICT)

2. Diseño de la capa de datos

2.1 Qué viaja a la tablet

La tablet necesita operar offline, pero el TP8 prohíbe datos personales en SQLite; por eso viaja solo el catálogo mínimo para armar un borrador.

Dato	¿Viaja?	En SQLite	Motivo
`id_producto`	Sí	`producto_cache.id`	Relaciona el borrador con el catálogo
`nombre`	Sí	`nombre`	Mostrar el producto al vendedor
`precio_lista`	Sí	`precio_centavos INTEGER`	Centavos enteros, no float; es "precio visto", no oficial
`stock`	Sí	`stock_visto`	Foto del momento, no reserva
`activo`	Sí	`eliminado` (0/1)	Baja lógica
Fecha de copia	Sí	`fecha_copia`	Detectar snapshot desactualizado
Clientes, credenciales, historial de pedidos, categorías	**No**	—	Datos personales o innecesarios; riesgo de exposición en un dispositivo que puede perderse

La fuente de verdad es PostgreSQL: la tablet nunca escribe precio ni stock en el servidor.

2.2 Mapeo PostgreSQL → SQLite

PostgreSQL	SQLite (`producto_cache`)	Regla
`id_producto`	`id`	Identificador original
`nombre`	`nombre`	Texto no vacío
`precio_lista NUMERIC(10,2)`	`precio_centavos INTEGER`	`ROUND(precio_lista * 100)::bigint` ; nunca float
`stock INTEGER`	`stock_visto INTEGER`	Solo lectura; `>= 0`
`stock > 0 AND activo`	`disponible INTEGER`	`0` / `1`
`NOT activo`	`eliminado INTEGER`	`0` / `1` (baja lógica)
Fecha de exportación	`fecha_copia TEXT`	ISO 8601

2.3 Snapshot con booleanos t/f y staging

La vista `v_snapshot_movil` (PostgreSQL) se exporta a CSV con `psql` ; en ese CSV los booleanos salen como `t` / `f` . La carga se hace en dos etapas: 1) `snapshot_servidor` , tabla de **staging** con tipos flexibles que absorbe el CSV tal como viene; 2) `producto_cache` , cargado con `CASE WHEN ... IN ('t','true','1') THEN 1 ELSE 0 END` y `CAST` a `INTEGER` .

Nunca se importa directo a `producto_cache` : es STRICT y sus `CHECK (disponible IN (0,1))` rechazarían `t` / `f` . El precio viaja en **centavos enteros** para evitar los errores de representación de punto flotante; SQLite STRICT rechaza un texto en una columna `INTEGER` .

2.4 Semáforo e idempotencia en la tablet

El semáforo (`semaforo.sql`) evalúa solo solicitudes en estado `PENDIENTE` contra el snapshot:

Color	Estado	Regla
Rojo	`RECHAZADO`	Producto inexistente, no disponible, eliminado o stock insuficiente
Amarillo	`CONFLICTO`	No hay rojo, pero cambió algún precio
Verde	`SIMULADO_APROBADO`	Todo coincide

Idempotencia: repetir la simulación no reevalúa (las solicitudes ya no están `PENDIENTE`) y la PK `solicitud_uuid` de `cola_sync` impide duplicados. En un sistema real, la protección vive del lado servidor: clave de idempotencia única + `ON CONFLICT` + log de solicitudes.

3. Implementación y evidencia

Entorno: el CLI de `sqlite3` no está instalado; los entregables son SQL puro y el arnés `run_pruebas.py` (Python 3.14 + SQLite 3.50.4, con soporte STRICT) los ejecuta. `PRAGMA foreign_keys = ON` se fuerza en cada conexión porque es un ajuste por conexión, no persistente en el archivo.

3.1 Cuatro tablas STRICT (Consigna 1)

`schema_sqlite.sql` crea 4 tablas STRICT con CHECK y claves foráneas: `producto_cache` (precio en centavos entero y no negativo, `disponible` / `eliminado` solo 0/1 y nunca ambos 1), `pedido_borrador` (`forma_pago` limitada a EFECTIVO/TARJETA/TRANSFERENCIA, usuario sin email, `id_local` tipo UUID), `item_borrador` (2 claves foráneas, `cantidad > 0`) y `cola_sync` (PK `solicitud_uuid` , 4 estados, estado final exige motivo).

3.2 Pruebas: 55 PASS, 0 FAIL

`python run_pruebas.py --db foodstore_movil.db --fresh` → **55 PASS, 0 FAIL** (2026-11-01, evidencia en `movil/salida_pruebas.txt`). Cubren:

- **C1:** catálogo disponible sin servidor (5 productos de prueba: Agua 500ml, Alfajor, Gaseosa 1.5L, Pan artesanal, Queso 500g).

- **Borrador con `BEGIN … COMMIT`:** total exacto de 220.050 centavos = $2.200,50; y operación deshecha con `ROLLBACK` sin filas residuales.
- **C7:** cantidad `0` (CHECK) y producto inexistente (FK) fallan sin dejar filas nuevas.
- **Triangulación:** precio negativo, texto en columna `INTEGER`, booleanos fuera de 0/1, forma de pago inválida, UUID inválido, ítem duplicado (PK) y `solicitud_uuid` repetido (C6).

3.3 Carga del snapshot (Consigna 2.a)

`v_snapshot_movil` verificada en PostgreSQL (modo single-user): **50.000 filas** sin errores. CSV `servidor_snapshot.csv`: 50.001 líneas (1 encabezado + 50.000 datos). Resultado de `run_cargar_snapshot.py`:

Métrica	Valor
Total productos en `producto_cache`	**50.000**
Disponibles (`disponible=1, eliminado=0`)	**44.732**
Eliminados (baja lógica)	**5.061**
Rango de precios (centavos)	50.012 – 499.996
Rango de stock visto	0 – 200

3.4 Semáforo: casos C2–C6 (Consignas 2.b y 2.c)

Ejecutados con `python simular_casos.py --db foodstore_movil.db`:

Caso	Escenario	Resultado real
C2	Cambio de precio del producto 3 (178.065 → 190.000 centavos)	**CONFLICTO**; al aceptar el nuevo precio la reevaluación da **SIMULADO_APROBADO**
C3	Solicitar 200 unidades con stock visto de 137	**RECHAZADO** — Stock insuficiente
C4	Producto dado de baja (`eliminado=1, disponible=0`)	**RECHAZADO**; la fila permanece en `producto_cache` (baja lógica, sin DELETE)
C5	Sin cambios en snapshot ni borrador	**SIMULADO_APROBADO** — Todo coincide
C6	Reenviar la misma `solicitud_uuid`	Error de PK (`UNIQUE constraint failed`); repetir la simulación no reevalúa — idempotencia

El borrador de prueba original quedó `RECHAZADO – Producto no disponible`: el producto 2 tiene `disponible=0` y `stock_visto=0` en el snapshot real, lo que demuestra que el semáforo compara contra datos reales, no sintéticos.

3.5 Verificación de no modificación en PostgreSQL

Consulta final sobre `tp_food_store`: **200.000 pedidos**, **700.410 detalles**, y los productos 1–3 idénticos al snapshot (stock/activo/precio comprobados uno por uno: p. ej. producto 3 = stock 137, activo, 1.780,65 = 178.065 centavos). La simulación es de **solo lectura**: ningún stock descontado, ningún precio cambiado, ninguna venta confirmada.

3.6 MongoDB (proyección de lectura)

Colección `pedidos` con **5 documentos** (`_id` Long 1–5), validador `$jsonSchema` activo y 6 índices. Permite consultar pedidos completos con líneas embebidas sin `JOIN`; no es fuente de verdad ni participa de la sincronización con la tablet.

4. Plan de mantenimiento y code review con IA

4.1 Plan de mantenimiento (resumen)

- **Respaldo:** PostgreSQL diario con `pg_dump` (verificación por conteos contra 200.000/50.000/20.000); MongoDB semanal con `mongodump` (verificación: 5 documentos, 6 índices, validador activo); SQLite por tablet al cierre de cada jornada (copiar el `.db` y `PRAGMA integrity_check`).

- **Restauración de un respaldo viejo:** las tablets **re-sincronizan siempre** el catálogo, el semáforo reevalúa los borradores `PENDIENTE` / `CONFLICTO` y los `SIMULADO_APROBADO` no se envían automáticamente.
- **Revisión de `CONFLICTO` / `RECHAZADO`:** al cierre de cada jornada, por el vendedor (sus solicitudes) y el administrador (todas las tablets); toda acción queda registrada en `cola_sync` como historial.

4.2 Code review con IA: 4 hallazgos (2 corregidos, 1 aceptado, 1 descartado)

Revisión con IA (OpenCode) sobre todos los scripts de la cursada (TP7 y TP8). Por regla de la consigna, al menos un hallazgo debía descartarse con fundamento (no se acepta todo lo que dice la IA):

#	Script	Hallazgo	Decisión
1	`migracion_mongodb.js` (TP7)	No idempotente: reejecutar falla por `_id` duplicado (E11000) y `NamespaceAlreadyExists`; el `db.pedidos.drop()` está comentado	**Corregido**
2	`consultas_postgresql.sql` (TP7)	La equivalencia PostgreSQL↔MongoDB se verificó solo sobre los 5 pedidos de prueba, no sobre los 200.000 reales	**Aceptado** (observación válida sobre el alcance)
3	`semaforo.sql` (TP8)	Un borrador **sin ítems** caería en el `ELSE` y quedaría `SIMULADO_APROBADO`: una orden vacía no debe aprobarse; ninguna de las 55 pruebas lo cubría	**Corregido**
4	`cargar_snapshot.sql` (TP8)	`DELETE FROM producto_cache` con FKs activas fallaría si `item_borrador` referencia productos	**Descartado con fundamento**

Fundamento del descarte (hallazgo 4): es una restricción de uso documentada, no un bug. El propio `cargar_snapshot.sql` indica que debe ejecutarse vía `run_cargar_snapshot.py`, que desactiva las FKs justo antes del `DELETE FROM producto_cache` (líneas 83–85 del arnés) y las reactiva después. El flujo previsto ya resuelve el caso.

Verificación: cada hallazgo se contrastó con las bases reales (MongoDB: 5 documentos con `_id` 1–5; SQLite: esquema y evidencia de las 55 pruebas; PostgreSQL: inspección de scripts y documentación). La bitácora `bitacora_ia.md` registra qué se aceptó y qué se descartó en cada interacción con la IA, y `DUIA.md` declara: todo código generado con IA fue leído, comprendido y probado; todo lo que no se puede explicar en la defensa cuenta como no hecho.

5. Conclusiones y mejoras futuras

Mandato cumplido: la arquitectura respeta la regla de oro. La tablet opera offline con una foto de solo lectura del catálogo y propone borradores; el semáforo local da una primera evaluación (rojo/amarillo/verde), pero la palabra final siempre es de PostgreSQL, donde se confirma la venta y se descuenta stock.

Evidencia clave: 55 PASS / 0 FAIL sobre 4 tablas STRICT; snapshot real de 50.000 productos (44.732 disponibles, 5.061 eliminados) sincronizado vía vista → CSV → staging → `producto_cache`; los 5 estados del semáforo comprobados con los casos C2–C6 (`CONFLICTO`, `RECHAZADO` ×2, `SIMULADO_APROBADO`, idempotencia por PK); PostgreSQL intacto tras la simulación (200.000 pedidos, 700.410 detalles).

Mejoras futuras:

1. **Idempotencia real del lado servidor:** implementar en PostgreSQL la clave de idempotencia derivada de `solicitud_uuid` con `ON CONFLICT` y log de solicitudes, dentro de la misma transacción que descuenta stock. La garantía actual vive solo en SQLite.
2. **Validación de borrador no vacío en el esquema** (continuidad del hallazgo 3) y cobertura del caso en `pruebas_sqlite.sql`.
3. **Reejecución idempotente de la migración a MongoDB** (hallazgo 1: descomentar `drop()` o envolver la creación en try/catch).
4. **Re-sincronización incremental** del snapshot y señal de "resync" del servidor hacia las tablets, formalizando el procedimiento del plan de mantenimiento.

Informe generado el 2026-11-01. Todas las cifras corresponden a ejecuciones reales documentadas en `docs/evidencias_consigna1.md` y `docs/evidencias_consigna2.md`.